

PRODUCT REQUIREMENTS DOCUMENT

IdeaForge AI

An AI-Powered Academic Project Intelligence Platform

CSE299 Capstone Project

Document Type: Product Requirements Document (PRD)

Version 1.0 | July 2026

Prepared for: University Supervisor & Development Team

Team Size: 4 Members

Revision History

Version	Date	Author	Description
0.1	Jun 2026	Team Lead	Initial draft based on project proposal
1.0	Jul 2026	Team Lead	Complete PRD for supervisor submission and team reference

Table of Contents

1. Executive Summary
2. Product Vision & Problem Statement
3. Business & Product Goals
4. Success Metrics (KPIs)
5. Stakeholders & User Personas
6. User Stories & User Journey
7. Functional Requirements
8. Non-Functional Requirements
9. Portal Feature Specifications
10. Authentication Flow
11. AI Features & RAG Architecture
12. System & Backend Architecture
13. Database Design
14. API Specifications
15. Coding Standards & Git Strategy
16. Risk Analysis & Security
17. Testing Strategy
18. Deployment Strategy
19. Team Roles & Responsibilities
20. 14-Week Agile Development Roadmap
21. Future Scope

1. Executive Summary

IdeaForge AI is an AI-powered Academic Project Intelligence Platform designed to help university students discover original, feasible capstone project ideas and to help faculty review proposals more efficiently. The platform combines a searchable repository of past academic projects with Retrieval-Augmented Generation (RAG) so that recommendations, similarity checks, and chatbot answers are grounded in real institutional history rather than generic AI output.

The system is organized into three portals — Student, Faculty, and Admin — and is being built by a four-person capstone team over a 14-week Agile schedule. This document defines the product's purpose, requirements, architecture, and delivery plan, and serves as the single source of truth for both the development team and the university supervisor.

Document Purpose

- Provide a shared, unambiguous definition of what will be built.
- Define functional and non-functional requirements with acceptance criteria.
- Describe the AI/RAG architecture, database design, and API contract.
- Assign clear weekly responsibilities to each of the four team members.

2. Product Vision & Problem Statement

2.1 Vision Statement

To become the intelligent academic companion that every student consults before starting a project — one that prevents duplicated work, surfaces feasible and original ideas, and gives faculty AI-assisted tools to evaluate proposals faster and more consistently.

2.2 Problem Statement

University project archives exist but are difficult to search and provide no active guidance. As a result:

- Students unknowingly repeat project ideas that already exist in the archive.
- Students have no reliable way to judge the difficulty or feasibility of an idea before committing.
- Faculty manually compare new proposals against prior work, which is slow and inconsistent.
- Students receive little guidance on suitable technology stacks or potential supervisors.
- There is no continuously growing, searchable knowledge base that gets smarter as more projects are added.

3. Business & Product Goals

3.1 Business Goals

- Deliver a working, demo-ready capstone project that satisfies CSE299 evaluation criteria.
- Reduce duplicate/near-duplicate project submissions within the pilot department.
- Reduce average faculty proposal-review time.
- Produce a reusable, extensible platform architecture that can scale to more departments.

3.2 Product Goals

- Let students search and discover prior projects using natural language, not just keywords.
- Automatically flag proposal similarity and novelty before submission.
- Recommend suitable technologies and potential supervisors based on proposal content.
- Give faculty AI-generated summaries and analytics to speed up review.
- Offer a RAG-based chatbot that answers project-related questions using the platform's own data.

4. Success Metrics (KPIs)

Metric	Target	Measured By
Semantic search relevance	Top-5 results judged relevant $\geq$ 80% of test queries	Manual evaluation set
Similarity detection accuracy	$\geq$ 85% agreement with manual faculty review on a test set	Faculty spot-check
Faculty review time per proposal	Reduced vs. baseline manual review	Before/after timing
Chatbot answer groundedness	$\geq$ 90% of answers traceable to retrieved sources	Manual RAG evaluation
System uptime (demo period)	$\geq$ 99%	Deployment monitoring
Core feature completion	100% of MVP scope delivered by Week 14	Sprint tracking

5. Stakeholders & User Personas

5.1 Stakeholders

Role	Interest
Students	Primary end users seeking project ideas and proposal feedback
Faculty Members	Review and approve/reject proposals, supervise projects
Department Admin	Manage accounts, departments, and the project archive
University Supervisor	Evaluates the capstone deliverable and documentation
Development Team (4 members)	Design, build, test, and deploy the platform

5.2 User Personas

Persona	Goals	Frustrations
Ayesha — 3rd Year CSE Student	Find an original, feasible project idea quickly	Doesn't know what's already been done
Dr. Rahman — Faculty Supervisor	Review proposals fast without missing duplicates	Manual comparison is tedious and error-prone
Admin Officer — Department Staff	Keep the archive and user accounts organized	No centralized system today

6. User Stories & User Journey

6.1 Key User Stories

ID	As a...	I want to...	So that...
US-01	Student	search past projects in plain language	I can see what has already been done
US-02	Student	submit a proposal and see a similarity score	I know if my idea is too close to an existing one
US-03	Student	get technology and supervisor suggestions	I can plan my project realistically
US-04	Student	ask the chatbot questions about past projects	I get quick, grounded answers
US-05	Faculty	see an AI-generated summary of each proposal	I can review faster
US-06	Faculty	view a similarity/novelty report per proposal	I can make an informed approve/reject decision
US-07	Admin	manage users, departments, and the archive	the platform stays accurate and up to date

6.2 Student User Journey (Simplified)

Register/Login -> Search Prior Projects -> Get AI Idea Suggestions
 -> Draft Proposal -> Run Similarity & Novelty Check -> Get Tech/
 Supervisor Recommendations -> Submit Proposal -> Track Status ->
 Receive Faculty Feedback -> (Ask Chatbot at any step)

7. Functional Requirements

ID	Requirement	Priority
FR-01	Users can register/login with role-based access (Student, Faculty, Admin)	High
FR-02	Students can perform semantic search over the project archive	High
FR-03	Students can submit proposals with title, abstract, and documents	High
FR-04	System computes a similarity score against archived projects	High
FR-05	System generates a novelty analysis and difficulty estimate	High
FR-06	System recommends technologies and supervisors per proposal	Medium
FR-07	Faculty can view, comment on, approve or reject proposals	High
FR-08	Faculty can view AI-generated proposal summaries	Medium
FR-09	Admin can manage users, departments, and archive entries	High
FR-10	RAG chatbot answers questions using retrieved project data	Medium
FR-11	System sends status notifications to students	Low
FR-12	Admin/Faculty can view analytics dashboards	Medium

8. Non-Functional Requirements

Category	Requirement
Performance	Semantic search results returned in under 2 seconds for a typical query
Scalability	Architecture supports growth from one department archive to multiple departments
Security	JWT-based authentication; role-based authorization on every endpoint
Reliability	Core APIs target 99% availability during evaluation/demo period
Usability	Responsive UI usable on desktop and mobile browsers
Maintainability	Modular FastAPI + React codebase with documented API contracts
Portability	Fully containerized with Docker for deployment on Render/Railway/VPS
Data Privacy	Student and faculty data access limited strictly by role

9. Portal Feature Specifications

9.1 Student Portal

Feature	Description	Acceptance Criteria
Auth & Profile	Register/login, manage profile	User can log in with valid JWT and see their profile
Semantic Search	Natural-language project search	Top results are ranked by semantic relevance, not just keyword match
Proposal Submission	Submit title, abstract, and files	Proposal is stored with status = 'Pending' and visible to the student
Similarity & Novelty Report	Shows similarity score + novelty summary	Score is generated and displayed before final submission
Recommendations	Tech stack & supervisor suggestions	At least 3 relevant suggestions returned per proposal
Chatbot	RAG Q&A on prior projects	Answers cite/reference retrieved project sources
Status Tracking	View proposal status & feedback	Status updates reflect faculty actions in real time

9.2 Faculty Portal

Feature	Description	Acceptance Criteria
Proposal Review Queue	List of assigned/pending proposals	Faculty sees all proposals awaiting their review
AI Summary	Auto-generated proposal summary	Summary reflects the actual proposal content
Similarity Report View	Detailed comparison to archive matches	Faculty can view matched projects and similarity %
Approve/Reject/Request Changes	Decision workflow with comments	Decision updates the proposal status and notifies the student
Analytics Dashboard	Department-level charts	Dashboard reflects current data on load

9.3 Admin Portal

Feature	Description	Acceptance Criteria
User Management	Create/edit/deactivate accounts	Only Admin role can access this feature
Department Management	Manage departments & sessions	Changes reflect immediately across portals
Archive Management	Add/remove/edit archived projects	Archive changes trigger re-indexing in the vector store
System Reports	Usage and activity reports	Reports are exportable and reflect real data

10. Authentication Flow

Authentication uses JWT access + refresh tokens issued by the FastAPI backend. Roles (Student, Faculty, Admin) are embedded as claims and checked by a dependency on every protected endpoint.

```
Client -> POST /auth/login (email, password)
Backend -> verify credentials against PostgreSQL (hashed password)
Backend -> issue JWT access token (short-lived) + refresh token
Client -> stores tokens, attaches access token as Bearer header
Backend -> on each request: decode JWT -> check role -> allow/deny
Client -> POST /auth/refresh when access token expires
```

11. AI Features & RAG Architecture

11.1 AI Feature Set

Feature	How It Works
Semantic Search	Query embedded via Sentence Transformers, matched against ChromaDB vectors
Similarity Detection	Cosine similarity between new proposal embedding and archive embeddings
Novelty Analysis	LLM compares top similar matches and explains what is/isn't novel
Difficulty Estimation	LLM scores complexity based on scope, techstack, and comparable past projects
Technology Recommendation	LLM suggests stack based on proposal requirements + retrieved similar projects
Supervisor Recommendation	Matches proposal topic against faculty research-interest embeddings
AI Proposal Summary	LLM generates a concise structured summary for faculty review
RAG Chatbot	Retrieves relevant chunks from ChromaDB, then LLM answers grounded in those chunks

11.2 RAG Pipeline

```

Documents (Past Projects/Proposals)
  -> Chunking -> Sentence Transformer Embeddings -> ChromaDB (Vector Store)

User Query -> Embed Query -> ChromaDB Similarity Search (top-k chunks)
  -> Build Context -> Prompt Template -> Gemini API (LLM)
  -> Grounded Response -> Returned to User

```

11.3 Prompt Engineering Strategy

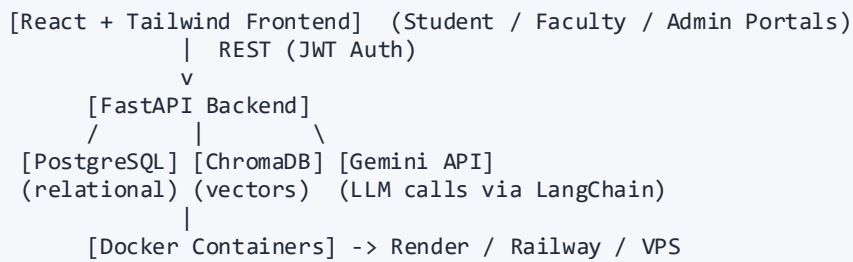
- Use structured prompt templates per feature (summary, novelty, recommendation, chatbot) rather than one generic prompt.
- Always inject retrieved context and explicitly instruct the model to answer only from that context where grounding matters (chatbot, novelty).
- Constrain output format (e.g., JSON) for features that feed the UI directly, such as difficulty scores and recommendation lists.
- Maintain a versioned prompt library so prompt changes can be tracked and evaluated.

11.4 Embedding & ChromaDB Notes

- Embedding model: Sentence Transformers (e.g., all-MiniLM-family) for speed and reasonable accuracy on academic text.
- One ChromaDB collection for archived projects, one for proposal drafts, to keep retrieval scoped and fast.
- Re-embedding is triggered whenever Admin adds/edits an archive entry or a student updates a proposal.

12. System & Backend Architecture

12.1 High-Level System Architecture



12.2 Backend Architecture

- FastAPI app organized by domain routers: auth, projects, proposals, ai, admin.
- SQLAlchemy models + Alembic-style migrations for PostgreSQL.
- A dedicated 'ai_service' layer wraps LangChain calls so routers stay thin.
- Background/async tasks handle embedding generation to avoid blocking API responses.

12.3 Frontend Architecture

- React.js with role-based routing (Student / Faculty / Admin layouts).
- TailwindCSS for styling; shared component library for cards, tables, and charts.
- Central API client (Axios) with interceptors for JWT attach/refresh.

13. Database Design

13.1 ER Diagram (Description)

Users (1) ---- (M) Proposals
Departments (1) ---- (M) Users
Proposals (1) ---- (1) SimilarityReports
Proposals (M) ---- (M) ArchivedProjects [via SimilarityMatches]
Users(Faculty) (1) ---- (M) ProposalReviews
Proposals (1) ---- (M) Feedback

13.2 Core Tables

Table	Key Columns
users	id, name, email, password_hash, role, department_id
departments	id, name, code
archived_projects	id, title, abstract, tech_stack, year, embedding_id
proposals	id, student_id, title, abstract, status, difficulty_score, created_at
similarity_reports	id, proposal_id, matched_project_id, similarity_score, novelty_summary
recommendations	id, proposal_id, type (tech/supervisor), value, confidence
reviews	id, proposal_id, faculty_id, decision, comments, reviewed_at
chat_sessions	id, user_id, created_at
chat_messages	id, session_id, sender, message, sources

14. API Specifications

Method	Endpoint	Description	Role
POST	/auth/register	Register a new user	Public
POST	/auth/login	Login, returns JWT tokens	Public
GET	/projects/search?q=	Semantic search over archive	Student
POST	/proposals	Submit a new proposal	Student
GET	/proposals/{id}/similarity	Get similarity/novelty report	Student/Faculty
GET	/proposals/{id}/recommendations	Get tech/supervisor suggestions	Student
GET	/faculty/proposals	List proposals for review	Faculty
POST	/faculty/proposals/{id}/review	Approve/reject/comment	Faculty
POST	/chat/message	Send a message to the RAG chatbot	Student/Faculty
GET	/admin/users	List/manage users	Admin
POST	/admin/archive	Add archived project entry	Admin

15. Coding Standards & Git Branching Strategy

15.1 Coding Standards

- Python: PEP8, type hints, Pydantic schemas for all request/response models.
- React: functional components + hooks, one component per file, Tailwind utility classes only.
- All endpoints documented via FastAPI's auto-generated OpenAPI docs.
- Meaningful commit messages following Conventional Commits (feat:, fix:, chore:, docs:).

15.2 Git Branching Strategy

- main — always deployable.
- develop — integration branch for completed features.
- feature/<name> — one branch per feature, merged into develop via pull request.
- Each PR requires at least one teammate review before merge.

16. Risk Analysis & Security

16.1 Risks & Mitigation

Risk	Impact	Mitigation
LLM API rate limits/cost	AI features slow or unavailable	Cache responses; add local fallback plan
Small archive limits search quality	Weak recommendations early on	Seed archive with sample past projects during Weeks 1-3
Similarity detection false positives	Faculty distrust the feature	Tune thresholds; always show evidence, not just a score
Team member availability/exams	Sprint delays	Buffer weeks 12-13 reserved for catch-up and QA
Scope creep	Missed deadline	MVP scope locked after Week 1; extras go to Future Scope

16.2 Security Considerations

- Passwords hashed (bcrypt); never stored or logged in plaintext.
- JWT access tokens short-lived; refresh tokens rotated.
- Role-based authorization enforced server-side on every endpoint, not just hidden in the UI.
- File uploads validated for type/size before storage.
- Environment secrets (DB credentials, Gemini API key) kept out of source control via .env.

17. Testing Strategy

Level	Approach	Owner
Unit Tests	Pytest for backend services, CRUD, and auth logic	Member 3
AI Evaluation	Manual evaluation set for search relevance, similarity accuracy, groundedness	Member 4
Integration Tests	End-to-end flow: submit proposal -> similarity -> review	Member 3 + Member 4
UI/Manual QA	Cross-browser, responsive checks on all three portals	Member 1 + Member 2
User Acceptance	Walkthrough with supervisor/sample users before final demo	Whole Team

18. Deployment Strategy

- All services containerized with Docker (frontend, backend, and a Postgres/Chroma service definition).
- docker-compose used for local development; same images deployed to Render/Railway/VPS.
- GitHub used for version control; deployment triggered from the main branch.
- Environment variables (DB URL, JWT secret, Gemini API key) configured per environment, never hardcoded.

```

GitHub (main branch) -> Build Docker Images -> Push
  -> Deploy: Frontend (static/CDN) + Backend (Render/Railway/VPS)
  -> PostgreSQL (managed) + ChromaDB (persistent volume)

```


19. Team Roles & Responsibilities

Member	Role	Core Ownership
Member 1	Frontend Developer — Student Portal	Auth UI, student dashboard, search UI, proposal submission/status UI, recommendation pages, chatbot UI, responsive design, API integration
Member 2	Frontend Developer — Faculty & Admin	Faculty/Admin dashboards, review & similarity report pages, analytics/charts, user & department management UI, feedback/settings UI
Member 3	Backend Developer	FastAPI project, JWT auth, REST APIs, CRUD, PostgreSQL integration, file upload, API docs, unit testing
Member 4	Technical Lead — AI & Architecture	LangChain/RAG, ChromaDB, semantic search, similarity/novelty/recommendation engines, chatbot, prompt engineering, Gemini integration, system architecture, security, DevOps, QA, documentation

20. 14-Week Agile Development Roadmap

The roadmap below lists, for every week, the sprint goal and the specific weekly goal for each of the four team members, followed by the shared deliverable and sprint milestone. This table is the team's single source of truth for weekly planning.

Wk	Sprint Goal	Member 1 (FE-Student)	Member 2 (FE-Faculty/Admin)	Member 3 (Backend)	Member 4 (AI/Tech Lead)
1	Project Setup & Planning	Repo setup, React project scaffold, UI kit selection	Wireframes for Faculty/Admin portals	FastAPI project scaffold, DB connection	Finalize architecture, DB schema draft, tool setup (LangChain, ChromaDB)
2	Auth Foundation	Login/Register UI	Wireframe review, Faculty/Admin auth UI	JWT auth API + user model	Review DB schema, set up ChromaDB instance
3	Core Dashboards	Student dashboard shell	Faculty & Admin dashboard shells	Core CRUD APIs (users, departments)	Embedding pipeline (Sentence Transformers) online
4	Search & Archive	Project search UI (student)	Proposal review list UI (faculty)	Archived project + proposal APIs	Semantic search engine (ChromaDB query) built
5	Proposal Submission	Proposal submission form + file upload UI	Similarity report UI (faculty view)	File upload API + proposal storage	Similarity detection module (cosine similarity)
6	AI Insights I	AI recommendation pages (student)	Analytics dashboard + charts (faculty/admin)	Notification & status-update APIs	Novelty analysis + difficulty estimation (LLM)
7	Integration Sprint 1	Connect student portal FE to BE fully	Connect faculty portal FE to BE fully	Support both integrations, fix API bugs	Technology recommendation engine
8	Chatbot & Admin UI	Chatbot UI (student)	User & department management UI (admin)	API documentation, backend unit tests	RAG chatbot backend (retrieval + Gemini)
9	Recommendations II	Wire supervisor recommendation into UI	Feedback & settings UI	Backend unit testing continued	Supervisor recommendation engine + prompt tuning
10	Polish & Hardening	Responsive design pass, student portal polish	Faculty/Admin UI polish	API security hardening, input validation	AI evaluation on test set, prompt refinement
11	Integration Sprint 2	Full regression pass — student flows	Full regression pass — faculty/admin flows	Fix cross-module integration bugs	End-to-end AI pipeline verification
12	QA Round 1 & Dockerization	Bug fixes from QA (frontend)	Bug fixes from QA (frontend)	Bug fixes from QA (backend)	Dockerize all services, docker-compose setup
13	QA Round 2 & Deployment Prep	Final UI polish + bug fixes	Final UI polish + bug fixes	Deployment config, environment variables	Performance tuning, deploy to Render/Railway/VPS
14	Final Deployment & Demo	Demo rehearsal, final UI check	Demo rehearsal, final UI check	Production deployment support	Final documentation, deployment guide, demo script

20.1 Phase-Level Risks & Dependencies

Phase	Focus	Key Risk	Key Dependency
Weeks 1-4	Foundation & Search	Schema changes mid-sprint; embedding pipeline setup delays	DB schema must be locked before Week 3 APIs begin

Phase	Focus	Key Risk	Key Dependency
Weeks 5-8	Core AI Features	LLM prompt quality issues; similarity thresholds need tuning	Archive needs sample data seeded by Week 5
Weeks 9-11	Integration	Merge conflicts across FE/BE; end-to-end bugs surface late	All portal features must be functionally complete by Week 10
Weeks 12-14	Hardening & Launch	Deployment environment differences from local Docker setup	Docker images must be stable before Week 13 deploy

21. Future Scope

- Integration with university management systems for automatic student/course data sync.
- Automatic research paper recommendations alongside past-project recommendations.
- GitHub repository analysis to assess implementation quality, not just proposal text.
- AI-generated implementation roadmaps for approved projects.
- Industry mentor portal for external project guidance.
- Native mobile application for students and faculty.
- Local/open-source LLM support as a fallback to the Gemini API.